

ASSIGNMENT – 2 (week 3 & 4)

Part I:

Question 1: In SQL, NULL refers to unknown or missing. Because it is unknown when the equal to operator sees NULL it does not return yes or no. It evaluates to UNKNOWN. So, when comparing phone numbers with NULL it evaluates to UNKNOWN for every single row, returning zero records.

Corrected Query:

```
SELECT COUNT(*)
```

```
FROM Student
```

```
WHERE phone_number IS NULL;
```

Question 2: Yes, the DISTINCT keyword here is redundant. As the GROUP BY clause already does the sorting of the departments by their unique names. The DISTINCT clause does not filter the unique departments again after it is sorted by the GROUP BY.

Question 3: The difference in the row count tells us that there is at least one customer in the CUSTOMER table who has never placed an order. The Scenario in which if there is customer with no orders in the CUSTOMER table they won't be displayed in the INNER JOIN as it would contain only the intersection of the CUSTOMER table and the ORDER table. Thus the LEFT JOIN returns more rows because it includes all customers who have never placed an order.

Question 4: Conceptually if SQL allowed the query, it would not know which specific employee_name to display for a department that has multiple employees. It would display a non-deterministic, arbitrary employee name from that department's group.

Question 5: The error is using the aggregate function AVG(SALARY) inside the where clause. Without the grouping of departments we cannot know the average salary at the WHERE stage.

Corrected Query:

```
SELECT department, AVG(SALARY)
```

```
FROM employee
```

```
GROUP BY department
```

```
HAVING AVG(SALARY) > 80000
```

Question 6:

- (a) Non-correlated subquery- The overall average product price is a single global constant value. It can be calculated once independently of the outer query's current row.
- (b) Correlated subquery – The subquery must refer to the specific department of the employee at each row and calculate the average salary of the employee's department.

Key Performance Implication:

Non-correlated subquery is executed exactly once whereas the correlated subquery is executed once for every single row in the table.

Part II:

(a)

```
SELECT c.customer_id, c.name, c.email  
FROM Customer c  
LEFT JOIN Order o ON c.customer_id = o.customer_id  
WHERE o.order_id IS NULL;
```

Using an inner join will return the values which are also present (not null) in the order table as well as on the customer table. The left join will give all the customer rows including the NULL and NOT NULL rows and using the WHERE clause will help us to filter the customers with zero orders.

(b)

```
SELECT r.restaurant_id, r.name, AVG(o_val.total_value) as avg_order_value  
FROM Restaurant as r  
JOIN (  
    SELECT o.restaurant_id, o.order_id, SUM(m.price * oi.quantity) as total_value  
    FROM Order o  
    JOIN OrderItem oi ON o.order_id = oi.order_id  
    JOIN MenuItem m ON oi.item_id = m.item_id  
    GROUP BY o.restaurant_id, o.order_id  
) o_val ON r.restaurant_id = o_val.restaurant_id  
GROUP BY r.restaurant_id, r.name  
HAVING COUNT(o_val.order_id) > 5;
```

(c)

```
SELECT r.restaurant_id, r.name, AVG(o_val.total_value) as avg_order_value
FROM Restaurant as r
JOIN (
    SELECT o.restaurant_id, o.order_id, SUM(m.price * oi.quantity) as total_value
    FROM Order o
    JOIN OrderItem oi ON o.order_id = oi.order_id
    JOIN MenuItem m ON oi.item_id = m.item_id
    GROUP BY o.restaurant_id, o.order_id
) o_val ON r.restaurant_id = o_val.restaurant_id
GROUP BY r.restaurant_id, r.name
HAVING COUNT(o_val.order_id) > 5
    AND AVG(o_val.total_value) > (
        SELECT SUM(m.price * oi.quantity) AS total_value
        FROM OrderItem oi
        JOIN MenuItem m ON oi.item_id = m.item_id
        GROUP BY oi.order_id
    ) all_orders
);
```

(d)

```
SELECT restaurant_id, item_id, item_name, total_quantity
FROM (
SELECT m.restaurant_id, m.item_id, m.item_name, SUM(oi.quantity) AS total_quantity,
RANK() OVER(PARTITION BY m.restaurant_id ORDER BY SUM(oi.quantity) DESC) AS
qty_rank
FROM MenuItem m
JOIN OrderItem oi ON m.item_id = oi.item_id
GROUP BY m.restaurant_id, m.item_id, m.item_name
) RankedItems
WHERE qty_rank = 1;
```

Part III:

Question 1:

- 1.) Student_id->student_name
- 2.) Course_id->course_name, instructor_id
- 3.) instructor_id->instructor_name, instructor_office
- 4.) (student_id, course_id) -> grade, enrolment_date

Question 2:

Yes, the table is in 1NF. A relation is in 1NF if all attribute values are atomic and there are no repeating groups or nested arrays

Question 3:

The primary key of the relation is the composite key (student_id, course_id). A 2NF violation occurs when a non-key attribute is dependent on only a part of the primary key.

Partial Dependencies:

Student_id -> student_name

Course_id -> course_name, instructor_id, instructor_name, instructor_office

Insertion anomaly : We cannot add a new course or its instructor to the database unless at least one student registers for it, because the primary key parts (student_id) cannot be null.

Deletion Anomaly : If the only student enrolled in a course drops it and we delete their row, we lose all information about that course's name, its instructor, and the instructor's office.

Update Anomaly: If a course name or instructor's office changes, we must search and update this value in every student's enrollment record.

Question 4:

A 3NF violation occurs when a non-key attribute determines another non-key attribute.

Transitive Dependency: (student_id, course_id) → course_id → instructor_id → instructor_office . Specifically, instructor_id → instructor_office is a transitive dependency because a non-prime attribute (instructor_office) is dependent on another non-prime attribute (instructor_id).

Update Anomaly: If an instructor changes offices, we must update the office location in every course record they teach.

Insertion Anomaly: We cannot record a new instructor's office location unless they are actively assigned to teach an existing course.

Question 5:

1. Student table: Student(student_id, student_name)
2. Instructor table: Instructor(instructor_id, instructor_name, instructor_office)
3. Course table: Course(course_id, course_name, instructor_id)
4. Enrollment table: Enrollment(student_id, course_id, grade, enrollment_date)

Part IV:

- In Part II (a), we could identify customers who have never placed an order using either a LEFT JOIN with a WHERE order_id IS NULL check, or a NOT EXISTS subquery. I chose the NOT EXISTS approach because of readability and correctness. NOT EXISTS is safer than NOT IN because if the subquery returns any NULL values, NOT IN evaluates to UNKNOWN and returns zero rows, whereas NOT EXISTS handles NULL values correctly.
- Normalizing the Enrollment table into 3NF removes data redundancy and prevents updates anomalies, but it trades off against query convenience. In the denormalized table, pulling a comprehensive list of a student's courses along with their instructors' names and offices is a simple, single-table scan. Every join operation incurs computation overhead from index lookups and join algorithms. For this reason, real-world systems sometimes intentionally keep some data denormalized to optimize read performance and simplify database queries.